

1 Introduction

This document provides a comprehensive, exhaustive guide to the CPDev-ESP32 Virtual Machine, with a specific focus on providing a level of detail sufficient for porting the VM to a new language or platform. It covers the VM's architecture, memory model, data types, a complete bytecode specification for every opcode, and the DCP file format.

Contents

1	Introduction	1
2	Architecture	2
3	Core VM and Execution Model	2
3.1	VM State and Registers	2
3.2	The Execution Cycle	2
3.3	Function Calling Convention	2
4	Memory Model	3
4.1	Program and Data Memory	3
4.2	Data Alignment and Access	3
4.3	The Data Offset Pointer (wDataOfs)	3
5	Data Types	3
6	Bytecode Specification	4
6.1	Instruction Format	4
6.2	Exhaustive Instruction Set Breakdown	4
6.2.1	Group 0x01: Addition (FG_ADD)	4
6.2.2	Group 0x02: Subtraction (FG_SUB)	4
6.2.3	Group 0x03: Multiplication (FG_MUL)	5
6.2.4	Groups 0x04: Division & Modulo (FG_DIV_MOD)	5
6.2.5	Group 0x05: Negation and NOT (FG_NEG_NOT)	5
6.2.6	Group 0x06: Math (FG_EXPT_ABS_DOW)	5
6.2.7	Group 0x10-0x15: Comparisons	6
6.2.8	Group 0x19: Type Conversions	6
6.2.9	Group 0x1C: System Control (FG_SYSCTRL)	6
6.2.10	Group 0x1E: Pointers (FG_POINTERS)	6
7	DCP File Specification	6
7.1	Purpose and Usage	7
7.2	XML Schema	7
7.3	Example	7
8	Platform Abstraction Layer (PAL)	7
8.1	Required Functions	7
8.2	Example Implementation (vm_arduino.cpp)	7
9	C++ Header Reference for Rust Port	7
9.1	Core Types and Definitions (vm.h)	8
9.1.1	Primitive Type Aliases	8
9.1.2	Core Data Structures	8
9.2	VM State and Registers (vmregisters.h)	8
9.3	Core VM Logic (vmtaskclass.h)	9
9.3.1	Public API Methods	9
9.3.2	Core Execution Logic (Protected)	9
9.3.3	Instruction Implementations (WMC_*) (Protected)	9
9.3.4	Internal Utilities (Protected)	10
9.3.5	Platform Abstraction Layer (VMP_*) (Virtual)	10
9.4	VMVariable and VMDCP Classes (vm_variable.h)	10
9.4.1	VMVariable Class Functions	10
9.4.2	VMDCP Class Functions	11
9.5	VMArduino Class Functions (vm_arduino.h)	11

2 Architecture

The VM is a stack-based virtual machine designed for resource-constrained embedded systems. It executes a custom bytecode instruction set to perform tasks typically found in PLC (Programmable Logic Controller) applications.

The main components are:

- **Core Execution Engine:** A loop that fetches, decodes, and executes bytecode instructions.
- **Program Memory:** Stores the bytecode of the application.
- **Data Memory:** A general-purpose memory region for variables, constants, and instance data.
- **Registers:** A set of registers to manage the VM's state, including the program counter and stack pointers.
- **Stacks:** Two stacks are used:
 - **Calling Stack:** Manages return addresses for function/function block calls.
 - **Data Offset Stack:** Manages the data offset pointer (`wData0fs`) for nested function block calls.
- **Platform Abstraction Layer (PAL):** A set of functions that must be implemented for a specific hardware platform to provide services like timing, I/O, and memory loading.

3 Core VM and Execution Model

The core logic is encapsulated in the `VMTaskClass` (from `vmtaskclass.h`).

3.1 VM State and Registers

The state of the VM at any given time is defined by the registers in `vmregisters.h`:

- `wProgramCounter`: The program counter (PC), holding the address of the next instruction in `pgmCode` to be executed.
- `wData0fs`: The data pointer register (DPTR). This is a crucial register that acts as a base address for accessing data within the current function block's context in `pgmData`. Most data access is relative to this pointer.
- `wCallingStack` & `wCallingStackPtr`: The stack and stack pointer for storing return addresses (`wProgramCounter` values) upon a `CALF`, `CALB`, or `CALBR` instruction.
- `wData0fsStack` & `wData0fsStackPtr`: The stack and stack pointer for storing the `wData0fs` of the caller. This allows nested function block calls to have their own data context.
- `bRunMode`: A byte indicating the current operational mode of the VM (e.g., `WM_MODE_NORMAL`, `WM_MODE_STOP`).
- `wStatus1`: A status word containing flags like `WMSTAT_CYCLEOVERRUN` or `WMSTAT_ARRAYBOUND`.
- `pgmCode`: A pointer to the beginning of the program memory (bytecode).
- `pgmData`: A pointer to the beginning of the data memory.
- `task_cycle`: The configured cycle time for the task in milliseconds.

3.2 The Execution Cycle

The VM's life cycle is managed by these key methods:

1. `WM_Initialize(mode)`: Sets up the VM for execution. It resets registers and stacks unless the `WM_MODE_CONTINUE` flag is set.
2. `WM_RunCycle()`: This is the heart of the VM. It performs one full scan (cycle) of the program.
 - It calls `VMP_PreCycle()` for any pre-cycle platform logic.
 - It enters a loop, calling `WM_RunCommand()` to execute instructions one by one.
 - The loop continues until `WM_RunCommand()` returns a non-zero result (e.g., `WM_TERMINATE` from a `TRML` instruction) or the `bRunMode` is cleared.
 - It calls `VMP_PostCycle()` for post-cycle logic, which typically includes calculating the cycle time and delaying to maintain a constant scan rate.
3. `WM_Shutdown()`: Called after the execution loop terminates. It sets the `WMSTAT_STOPPED` flag and calls the `VMP_PostProgram()` PAL function for cleanup.

3.3 Function Calling Convention

The VM supports function (`CALF`) and function block (`CALB`, `CALBR`) calls.

- **CALF (Call Function) / CALB (Call Function Block):**
 1. The current `wData0fs` is pushed onto the data offset stack.

2. The current `wProgramCounter` (pointing to the instruction after the call) is pushed onto the calling stack.
 3. The new `wData0fs` is calculated by adding the instance data offset (an operand of the call) to the current `wData0fs`.
 4. The `wProgramCounter` is set to the address of the function/block being called.
- **RETFB (Return from Function Block):**
 1. The `wProgramCounter` is restored by popping from the calling stack.
 2. The `wData0fs` is restored by popping from the data offset stack.

This mechanism allows for re-entrant code and proper encapsulation of function block instance data.

4 Memory Model

4.1 Program and Data Memory

The VM uses two distinct memory areas:

- **Program Memory (`pgmCode`):** A read-only area holding the bytecode. It is loaded at startup by the `VMP_LoadProgramAndData` or `VMP_LoadProgramFromArray` PAL function.
- **Data Memory (`pgmData`):** A read-write area for all variables. This includes global variables, local variables, function block instances, and temporary values.

4.2 Data Alignment and Access

The VM can be compiled with different memory access strategies, defined in `vm_cfg.h`:

- **Direct Access (default):** If `BYTE_ACCESS` and `MEMCPY_ACCESS` are undefined, the VM accesses multi-byte values by casting pointers directly. This is the fastest method but requires that data is properly aligned in memory.
- **BYTE_ACCESS:** If defined, the VM accesses multi-byte values byte-by-byte and reconstructs them. This handles unaligned access correctly but is slower.
- **MEMCPY_ACCESS:** If defined, the VM uses `memcpy` for all multi-byte data access. This is another way to handle unaligned access and can be safer on some architectures.

The data access functions (`getINT`, `setINT`, `getREAL`, etc.) are defined in `vm_data_access.h` and implemented in `vm_data_access.cpp`.

4.3 The Data Offset Pointer (`wData0fs`)

All data access within the VM is relative. An instruction does not contain the absolute address of a variable in `pgmData`. Instead, it contains an offset. The effective address is calculated as:

$$\text{effective_address} = \text{pgmData} + \text{wData0fs} + \text{instruction_offset}$$

The `wData0fs` points to the base of the current data context (i.e., the start of the instance data for the currently executing function block). This makes function blocks re-entrant, as each instance gets its own data area, and the code within the block accesses it through the `wData0fs` register. For global variables, `wData0fs` is typically 0.

5 Data Types

All data types are defined in `vm.h`.

- `WM_BOOL`, `WM_BYTE`, `WM_SINT`: 8-bit types.
- `WM_WORD`, `WM_INT`: 16-bit types.
- `WM_DWORD`, `WM_DINT`, `WM_TIME`: 32-bit types.
- `WM_LWORD`, `WM_LINT`: 64-bit types (requires `VM_LONG_SUPPORT`).
- `WM_REAL`: 32-bit float (requires `VM_REAL_SUPPORT`).
- `WM_LREAL`: 64-bit double (requires `VM_LREAL_SUPPORT`).
- `WM_ADDRESS`: 16-bit or 32-bit, depending on `VM_ADDRESSING_32`.
- `WM_DATE_AND_TIME`: A structure for holding calendar date and time (requires `VM_DATETIME_SUPPORT`).
- `WM_STRING`: A structure containing a length, buffer size, and a pointer to the character data (requires `VM_STRING_SUPPORT`).

6 Bytecode Specification

The bytecode is defined in `vmspec/vmdef.h`.

Where the number of operands N is encoded in the instruction option, it is represented as N in the opcode column (e.g., `0x01N1`).

6.1 Instruction Format

Each instruction is a 16-bit word (`WM_WORD`), composed of a group and an option.

- **Low Byte (Bits 0-7):** The instruction group (`FG_...`). This determines the general category of the operation and which `WMC_...` handler function is called by the VM.
- **High Byte (Bits 8-15):** The instruction-specific option (`wCmdOpt`). This byte is passed to the handler function to specify the exact operation, data types, or number of parameters.

Decoding `wCmdOpt` The meaning of the option byte depends on the instruction group:

- For many arithmetic, logical, and comparison groups, the high nibble (`0xN0`) specifies the number of input operands (N), while the low nibble (`0x0T`) specifies the data type (T).
- For system control, pointer, and conversion groups, the entire byte often serves as a unique opcode for a specific function.

6.2 Exhaustive Instruction Set Breakdown

6.2.1 Group `0x01`: Addition (`FG_ADD`)

Performs `dst = src1 + src2 + ... + srcN`. The number of source operands (N) is encoded in the high nibble of the opcode.

Opcode	Mnemonic	Operands	Description
<code>0x01N1</code>	<code>VMF_ADD_SINT</code>	<code>dst:SINT*, srcs:SINT*...</code>	Adds N SINT values.
<code>0x01N2</code>	<code>VMF_ADD_INT</code>	<code>dst:INT*, srcs:INT*...</code>	Adds N INT values.
<code>0x01N3</code>	<code>VMF_ADD_DINT</code>	<code>dst:DINT*, srcs:DINT*...</code>	Adds N DINT values.
<code>0x01N4</code>	<code>VMF_ADD_LINT</code>	<code>dst:LINT*, srcs:LINT*...</code>	Adds N LINT values.
<code>0x01N5</code>	<code>VMF_ADD_BYTE</code>	<code>dst:BYTE*, srcs:BYTE*...</code>	Adds N BYTE values.
<code>0x01N6</code>	<code>VMF_ADD_WORD</code>	<code>dst:WORD*, srcs:WORD*...</code>	Adds N WORD values.
<code>0x01N7</code>	<code>VMF_ADD_DWORD</code>	<code>dst:DWORD*, srcs:DWORD*...</code>	Adds N DWORD values.
<code>0x01N8</code>	<code>VMF_ADD_LWORD</code>	<code>dst:LWORD*, srcs:LWORD*...</code>	Adds N LWORD values.
<code>0x01N9</code>	<code>VMF_ADD_REAL</code>	<code>dst:REAL*, srcs:REAL*...</code>	Adds N REAL values.
<code>0x01NA</code>	<code>VMF_ADD_LREAL</code>	<code>dst:LREAL*, srcs:LREAL*...</code>	Adds N LREAL values.
<code>0x01NB</code>	<code>VMF_ADD_TIME</code>	<code>dst:TIME*, srcs:TIME*...</code>	Adds N TIME values.
<code>0x01NF</code>	<code>VMF_ADD_STRING</code>	<code>dst:STRING*, srcs:STRING*...</code>	Concatenates N STRING values.

6.2.2 Group `0x02`: Subtraction (`FG_SUB`)

Performs `dst = src1 - src2`.

Opcode	Mnemonic	Operands	Description
<code>0x0201</code>	<code>VMF_SUB_SINT_SINT</code>	<code>dst:SINT*, src1:SINT*, src2:SINT*</code>	Subtracts two SINT values.
<code>0x0202</code>	<code>VMF_SUB_INT_INT</code>	<code>dst:INT*, src1:INT*, src2:INT*</code>	Subtracts two INT values.
<code>0x0203</code>	<code>VMF_SUB_DINT_DINT</code>	<code>dst:DINT*, src1:DINT*, src2:DINT*</code>	Subtracts two DINT values.
<code>0x0204</code>	<code>VMF_SUB_LINT_LINT</code>	<code>dst:LINT*, src1:LINT*, src2:LINT*</code>	Subtracts two LINT values.
<code>0x0205</code>	<code>VMF_SUB_BYTE_BYTE</code>	<code>dst:BYTE*, src1:BYTE*, src2:BYTE*</code>	Subtracts two BYTE values.
<code>0x0206</code>	<code>VMF_SUB_WORD_WORD</code>	<code>dst:WORD*, src1:WORD*, src2:WORD*</code>	Subtracts two WORD values.
<code>0x0207</code>	<code>VMF_SUB_DWORD_DWORD</code>	<code>dst:DWORD*, src1:DWORD*, src2:DWORD*</code>	Subtracts two DWORD values.

0x0208	VMF_SUB_LWORD_LWORD	dst:LWORD*, src1:LWORD*, src2:LWORD*	Subtracts two LWORD values.
0x0209	VMF_SUB_REAL_REAL	dst:REAL*, src1:REAL*, src2:REAL*	Subtracts two REAL values.
0x020A	VMF_SUB_LREAL_LREAL	dst:LREAL*, src1:LREAL*, src2:LREAL*	Subtracts two LREAL values.
0x020B	VMF_SUB_TIME_TIME	dst:TIME*, src1:TIME*, src2:TIME*	Subtracts two TIME values.

6.2.3 Group 0x03: Multiplication (FG_MUL)

Performs $dst = src1 * src2 * \dots * srcN$.

Opcode	Mnemonic	Operands	Description
0x03N1	VMF_MUL_SINT	dst:SINT*, srcs:SINT*...	Multiplies N SINT values.
0x03N2	VMF_MUL_INT	dst:INT*, srcs:INT*...	Multiplies N INT values.
0x03N3	VMF_MUL_DINT	dst:DINT*, srcs:DINT*...	Multiplies N DINT values.
0x03N4	VMF_MUL_LINT	dst:LINT*, srcs:LINT*...	Multiplies N LINT values.
0x03N5	VMF_MUL_BYTE	dst:BYTE*, srcs:BYTE*...	Multiplies N BYTE values.
0x03N6	VMF_MUL_WORD	dst:WORD*, srcs:WORD*...	Multiplies N WORD values.
0x03N7	VMF_MUL_DWORD	dst:DWORD*, srcs:DWORD*...	Multiplies N DWORD values.
0x03N8	VMF_MUL_LWORD	dst:LWORD*, srcs:LWORD*...	Multiplies N LWORD values.
0x03N9	VMF_MUL_REAL	dst:REAL*, srcs:REAL*...	Multiplies N REAL values.
0x03NA	VMF_MUL_LREAL	dst:LREAL*, srcs:LREAL*...	Multiplies N LREAL values.

6.2.4 Groups 0x04: Division & Modulo (FG_DIV_MOD)

Performs $dst = src1 / src2$ or $dst = src1 \% src2$.

Opcode	Mnemonic	Operands	Description
0x0402	VMF_DIV_INT_INT	dst:INT*, dividend:INT*, divisor:INT*	Divides two INT values.
0x0412	VMF_MOD_INT_INT	dst:INT*, dividend:INT*, divisor:INT*	Modulo of two INT values.

6.2.5 Group 0x05: Negation and NOT (FG_NEG_NOT)

Opcode	Mnemonic	Operands	Description
0x0502	VMF_NEG_INT	dst:INT*, src:INT*	Performs arithmetic negation (-src).
0x0510	VMF_NOT_BOOL	dst:BOOL*, src:BOOL*	Performs logical NOT (!src).
0x0512	VMF_NOT_WORD	dst:WORD*, src:WORD*	Performs bitwise NOT (~src).

6.2.6 Group 0x06: Math (FG_EXPT_ABS_DOW)

Opcode	Mnemonic	Operands	Description
0x0607	VMF_EXPT_REAL_REAL	dst:REAL*, base:REAL*, exp:REAL*	Raises base to the power of exp.
0x0612	VMF_ABS_INT	dst:INT*, src:INT*	Calculates the absolute value of src.
0x0620	VMF_SQRT_REAL	dst:REAL*, src:REAL*	Calculates the square root of src.
0x0622	VMF_LN_REAL	dst:REAL*, src:REAL*	Calculates the natural logarithm of src.
0x0628	VMF_SIN_REAL	dst:REAL*, src:REAL*	Calculates the sine of src.
0x0640	VMF_GET_DAYOFWEEK_DATE	dst:INT*, src:DATE*	Calculates the day of the week from a DATE value.

6.2.7 Group 0x10-0x15: Comparisons

Result is a BOOL written to dst.

Opcode	Mnemonic	Operands	Description
0x1002	VMF_GT_INT_INT	dst:BOOL*, src1:INT*, src2:INT*	TRUE if src1 > src2.
0x1102	VMF_GE_INT_INT	dst:BOOL*, src1:INT*, src2:INT*	TRUE if src1 >= src2.
0x1202	VMF_EQ_INT_INT	dst:BOOL*, src1:INT*, src2:INT*	TRUE if src1 == src2.
0x1302	VMF_LE_INT_INT	dst:BOOL*, src1:INT*, src2:INT*	TRUE if src1 <= src2.
0x1402	VMF_LT_INT_INT	dst:BOOL*, src1:INT*, src2:INT*	TRUE if src1 < src2.
0x1502	VMF_NE_INT_INT	dst:BOOL*, src1:INT*, src2:INT*	TRUE if src1 != src2.

6.2.8 Group 0x19: Type Conversions

Opcode	Mnemonic	Operands	Description
0x1900	VMF_INT_TO_REAL	dst:REAL*, src:INT*	Converts INT to REAL.
0x190C	VMF_INT_TO_DINT	dst:DINT*, src:INT*	Converts INT to DINT.
0x190A	VMF_REAL_TO_INT	dst:INT*, src:REAL*	Truncates REAL to INT.

6.2.9 Group 0x1C: System Control (FG_SYSCtrl)

Opcode	Mnemonic	Operands	Description
0x1C00	VMF_JMP	target_addr	Unconditionally sets PC to target_addr.
0x1C01	VMF_JNZ	cond_addr, target_addr	Jumps to target_addr if the BOOL at cond_addr is TRUE.
0x1C02	VMF_JZ	cond_addr, target_addr	Jumps to target_addr if the BOOL at cond_addr is FALSE.
0x1C16	VMF_CALB	inst_offset, code_addr	Calls a Function Block. Pushes PC and DPTR. PC = code_addr, DPTR = old_DPTR + inst_offset.
0x1C18	VMF_RETFB		Returns from a Function Block. Pops PC and DPTR.
0x1C1D	VMF_TRML	addr	Terminates the current cycle and sets PC to addr for the start of the next cycle.
0x1C1F	VMF_MEMCP	dst_addr, src_addr, count_word	Copies count_word bytes from src_addr to dst_addr in data memory.

6.2.10 Group 0x1E: Pointers (FG_POINTERS)

Opcode	Mnemonic	Operands	Description
0x1E00	VMF_DPST0	dst_addr:ADDRESS*	Stores the current Data Pointer (wDataOfs) into the ADDRESS variable at dst_addr.
0x1E03	VMF_CALBR	inst_ptr_addr:ADDRESS*, code_addr	Call By Reference. Calls a function block where the instance address is read from the ADDRESS variable at inst_ptr_addr.
0x1E04	VMF_JNUL	ptr_addr:ADDRESS*, target_addr	Jumps to target_addr if the ADDRESS variable at ptr_addr is NULL (0).

7 DCP File Specification

The DCP (Device Configuration Project) file is an XML file that describes the global variables of a project. It allows the VM to resolve variable names to memory addresses.

7.1 Purpose and Usage

The VMDCP class in `vm_variable.cpp` parses this file. It is used to get the address and size of a variable by its name (LName), which is essential for external tools or an HMI to interact with the running application without hardcoded memory offsets.

7.2 XML Schema

The structure is as follows, based on `vm_variable.cpp` and `data/WeJeStSt.xml`.

```
<CPDEV>
  <TARGET>
    <GLOBAL>
      <VAR LName="VariableName" Addr="HexAddress" Size="ByteSize" Type="DataTypeName" />
      <!-- More VAR elements... -->
    </GLOBAL>
  </TARGET>
</CPDEV>
```

- **<CPDEV>**: The root element.
- **<TARGET>**: Contains the target-specific configuration.
- **<GLOBAL>**: Contains the list of global variables.
- **<VAR>**: Describes a single variable with the following attributes:
 - LName: The logical name of the variable (e.g., "MAIN.MyVar"). This is the primary identifier.
 - Addr: The hexadecimal memory offset of the variable within the global data area (0x...).
 - Size: The size of the variable in bytes.
 - Type: A string representing the data type (e.g., "BOOL", "INT").

7.3 Example

From `data/WeJeStSt.xml`:

```
<VAR LName="WeJeStSt.Wejscia.WE1" Addr="0x0" Size="1" Type="BOOL" />
```

This defines a BOOL variable named `WeJeStSt.Wejscia.WE1` located at address `0x0` in the global data area, and it is 1 byte in size. The `VMDCP::InitVariable("WeJeStSt.Wejscia.WE1")` method would return a `VMVariable` object with address 0 and size 1.

8 Platform Abstraction Layer (PAL)

To port the VM to a new platform, you must provide implementations for the pure virtual functions defined in `VMTaskClass` and the functions declared in `vmplatform.h`.

8.1 Required Functions

- `int VMP_LoadProgramAndData(const char* file, int datasize)`: Must allocate memory for `pgmCode` and `pgmData` and load the bytecode from the specified source.
- `WM_TIME VMP_CurrentTime()`: Must return a monotonic time value in milliseconds.
- `WM_REAL VMP_GetRandom()`: Must return a random float between 0.0 and 1.0.
- `void VMP_ReadRTC(WM_DATE_AND_TIME *dt)`: Must populate the `dt` struct with the current real-time clock value.
- Other optional functions for debugging, pre/post cycle hooks, and native function block execution.

8.2 Example Implementation (`vm_arduino.cpp`)

The `VMArduino` class provides a reference implementation. For example, `VMP_LoadProgramAndData` is implemented to load a file from an SD card (if `USE_SD_XCP` is defined), and `VMP_CurrentTime` is implemented using the Arduino `millis()` function. A developer porting the VM would replace this logic with the equivalent for their target OS or platform (e.g., using `fopen/fread` and `clock_gettime`).

9 C++ Header Reference for Rust Port

This section provides a detailed breakdown of the core C++ header files to guide the process of rewriting the VM in another language.

9.1 Core Types and Definitions (vm.h)

This file defines the fundamental data types used throughout the VM.

9.1.1 Primitive Type Aliases

C++ Type	Description
WM_BOOL	8-bit boolean (uint8_t).
WM_BYTE, WM_USINT	8-bit unsigned integer (uint8_t).
WM_SINT	8-bit signed integer (int8_t).
WM_WORD, WM_UINT	16-bit unsigned integer (uint16_t).
WM_INT	16-bit signed integer (int16_t).
WM_DWORD, WM_UDINT, WM_TIME	32-bit unsigned integer (uint32_t). WM_TIME is used for time durations in milliseconds.
WM_DINT	32-bit signed integer (int32_t).
WM_LWORD, WM_ULINT	64-bit unsigned integer (uint64_t), if VM_LONG_SUPPORT is enabled.
WM_LINT	64-bit signed integer (int64_t), if VM_LONG_SUPPORT is enabled.
WM_REAL	32-bit float, if VM_REAL_SUPPORT is enabled.
WM_LREAL	64-bit double, if VM_LREAL_SUPPORT is enabled.
WM_ADDRESS	VM's address type. WM_WORD (16-bit) or WM_DWORD (32-bit) depending on the VM_ADDRESSING_32 flag.

9.1.2 Core Data Structures

Struct	Description
WM_DATE_AND_TIME	A structure holding date and time-of-day components. Used for RTC functionality.
WM_STRING	Represents a string with a length, buffer size, and a pointer to the character data (chars).

9.2 VM State and Registers (vmregisters.h)

This file is not a typical header; it is directly included within the VMCLASS_NAME class definition in vmtaskclass.h. Its contents define the member variables that constitute the VM's state (its registers).

Register (Member Variable)	Description
pgmCode	WM_BYTE* pointer to the start of the loaded program bytecode memory.
pgmData	WM_BYTE* pointer to the start of the read/write data memory.
wProgramCounter	WM_ADDRESS that holds the address of the next instruction in pgmCode to be executed (Program Counter).
wDataOfs	WM_ADDRESS that acts as a base offset into pgmData for the current function block's instance data (Data Pointer).
wCallingStack[]	An array of WM_ADDRESS used as the call stack to store return addresses (wProgramCounter values) for function calls.
wCallingStackPtr	WM_BYTE index for the top of wCallingStack.
wDataOfsStack[]	An array of WM_ADDRESS used as a stack to store wDataOfs values from callers.
wDataOfsStackPtr	WM_BYTE index for the top of wDataOfsStack.
bRunMode	WM_BYTE flag indicating the VM's operational state (e.g., WM_MODE_NORMAL, WM_MODE_STOP).
wStatus1	WM_WORD bitfield for VM status flags (e.g., WMSTAT_CYCLEOVERRUN, WMSTAT_STOPPED).

task_cycle	WM_WORD configured target cycle time in milliseconds.
cycle_start_adr	WM_ADDRESS of the first instruction to run in a cycle. TRML instruction updates this.
nCycles	unsigned long counter for the number of execution cycles performed.
bResult	WM_BYTE flag used internally by the instruction execution loop.

9.3 Core VM Logic (vmtaskclass.h)

This file defines VMCLASS_NAME (e.g., VMTaskClass16), the central class containing the VM's execution logic, state, and platform interface.

9.3.1 Public API Methods

These are the primary functions for controlling the VM from outside.

Function	Description
WM_Run(int mode)	Starts the main execution loop of the VM.
WM_Stop(void)	Stops the VM's execution loop.
WM_GetData(...)	Reads a block of bytes from the VM's pgmData memory.
WM_SetData(...)	Writes a block of bytes to the VM's pgmData memory.
WM_GetCycles()	Returns the total number of cycles executed (nCycles).
WM_GetStatus1()	Returns the wStatus1 status word.

9.3.2 Core Execution Logic (Protected)

These methods form the heart of the VM's fetch-decode-execute cycle.

Function	Description
WM_Initialize(int mode)	Sets up the VM state before execution begins. Calls WM_InitRegisters() and clear_Stacks().
WM_RunCycle()	Executes a single scan/cycle. It repeatedly calls WM_RunCommand() until the program terminates the cycle (e.g., via TRML).
WM_Shutdown()	Performs cleanup after the main execution loop in WM_Run has finished.
WM_InitRegisters(void)	Resets all VM registers to their default initial state.
int WM_RunCommand(void)	Fetches one 16-bit instruction from pgmCode, decodes the instruction group, and calls the appropriate WMC_* handler function.

9.3.3 Instruction Implementations (WMC_*) (Protected)

These are the handler functions for the different bytecode instruction groups. They are called by WM_RunCommand.

Function	Description
WMC_ADD	Handles addition instructions.
WMC_SUB	Handles subtraction instructions.
WMC_MUL	Handles multiplication instructions.
WMC_DIV_MOD	Handles division and modulo instructions.
WMC_AND, WMC_OR, WMC_XOR	Handles bitwise logical operations.
WMC_GT, WMC_GE, WMC_LT, WMC_LE, WMC_EQ, WMC_NE	Handles comparison instructions.
WMC_CONV	Handles type conversion instructions.
WMC_SYSCTRL	Handles system control instructions like JMP, JNZ, CALB, RETFB, TRML.

WMC_Pointers	Handles pointer-related instructions like DPST0 and CALBR.
...	...and many others, corresponding to the instruction groups in vmdef.h.

9.3.4 Internal Utilities (Protected)

These are helper functions for common internal tasks.

Function Group	Description
GetProgramWord, GetProgramDWord, GetProgramAddress	Reads the next 2, 4, or sizeof(WM_ADDRESS) bytes from the pgmCode stream and increments the wProgramCounter. Used for fetching instruction operands.
getREAL, setREAL, getINT, setINT, ...	Functions for reading and writing different data types to/from the pgmData memory at a given address. These functions handle memory alignment issues based on compiler flags (BYTE_ACCESS, MEMCPY_ACCESS).
push_DataOfsStack, pull_DataOfsStack	Pushes/pops a value onto/from the wDataOfsStack and manages wDataOfsStackPtr.
push_CallingStack, pull_CallingStack	Pushes/pops a value onto/from the wCallingStack and manages wCallingStackPtr.
clear_Stacks	Resets both stack pointers to 0.

9.3.5 Platform Abstraction Layer (VMP_*) (Virtual)

These virtual functions define the interface that a platform-specific implementation must provide.

Function	Required?	Description
VMP_LoadProgramAndData(...)	Required (Pure Virtual)	Loads bytecode and allocates memory.
VMP_CurrentTime()	Required (Pure Virtual)	Returns system time in milliseconds.
VMP_GetRandom()	Required (Pure Virtual)	Returns a random float.
VMP_PreCycle, VMP_PostCycle	Optional	Hooks at the start and end of an execution cycle.
VMP_ReadRTC, VMP_WriteRTC	Optional	Functions for interacting with a Real-Time Clock.
VMP_Debug(...)	Optional	Callback for handling VM errors and debug events.
VMP_ExecNativeBlock(...)	Optional	Hook for executing custom function blocks written in native C++ code.

9.4 VMVariable and VMDCP Classes (vm_variable.h)

These classes are responsible for parsing the Device Configuration Project (DCP) file and providing access to VM variables by name.

9.4.1 VMVariable Class Functions

Function	Description
VMVariable(const char* name, WM_ADDRESS address, WM_BYTE size)	Constructor: Initializes a VMVariable object with a name, memory address, and size.
WM_ADDRESS GetAddress()	Returns the memory address of the variable.

WM_BYTE GetSize()	Returns the size of the variable in bytes.
-------------------	--

9.4.2 VMDCP Class Functions

Function	Description
int Load(const char* file)	Loads and parses the DCP XML file from a given file path. Returns 0 on success, or an error code.
int Load(File file)	Loads and parses the DCP XML file from an Arduino File object (platform-specific).
VMVariable* InitVariable(const char* name)	Searches for a variable by its logical name within the loaded DCP data and returns a pointer to a VMVariable object if found.

9.5 VMArduino Class Functions (vm_arduino.h)

The VMArduino class serves as a concrete implementation of the VMTaskClass abstract base class specifically for Arduino/ESP32 platforms. It overrides and provides implementations for the virtual PAL functions.

Function	Description
VMArduino()	Constructor: Initializes the VMArduino instance.
int VMP_LoadProgramAndData(const char* file, int datasize)	Overrides VMTaskClass::VMP_LoadProgramAndData to load program and data, often from SD card or internal flash.
int VMP_LoadProgramFromArray(const unsigned char* code, int datasize)	Additional method to load program and data from a byte array in memory.
void VMP_PostCycle(void)	Overrides VMTaskClass::VMP_PostCycle to implement cycle time management (e.g., delays).
void VMP_PreCycle(void)	Overrides VMTaskClass::VMP_PreCycle for any pre-cycle setup specific to Arduino.
void VMP_Debug(WM_BYTE err, WM_WORD aux)	Overrides VMTaskClass::VMP_Debug for platform-specific debugging output.
void VMP_PreProgram(void)	Overrides VMTaskClass::VMP_PreProgram for Arduino-specific initialization before program start.
void VMP_PreNextCommand()	Overrides VMTaskClass::VMP_PreNextCommand for any pre-command logic.
WM_TIME VMP_CurrentTime()	Overrides VMTaskClass::VMP_CurrentTime to return time using millis() or similar Arduino function.
void VMP_ReadRTC(WM_DATE_AND_TIME *dt)	Overrides VMTaskClass::VMP_ReadRTC for reading the real-time clock on Arduino.
WM_REAL VMP_GetRandom()	Overrides VMTaskClass::VMP_GetRandom to provide a random number generator using random() or similar Arduino function.
int WM_GetData(VMVariable* var, WM_BYTE* buf)	Overload of VMTaskClass::WM_GetData to read data using a VMVariable object.
WM_BYTE WM_GetDataByte(VMVariable* var)	Reads a single byte of data using a VMVariable object.
int WM_SetData(VMVariable* var, WM_BYTE* buf)	Overload of VMTaskClass::WM_SetData to write data using a VMVariable object.
int WM_SetDataByte(VMVariable* var, WM_BYTE value)	Writes a single byte of data using a VMVariable object.